UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

FACULTAD DE INGENIERIA DE PRODUCCION Y SERVICIOS

ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN



Asignatura: Ingenieria de software

Tema: Practica 5

Docente: Edgard Sarmiento Calizaya

Estudiantes:

- Condori Pallardelli, Emilio Alejandro

Arequipa - Perú

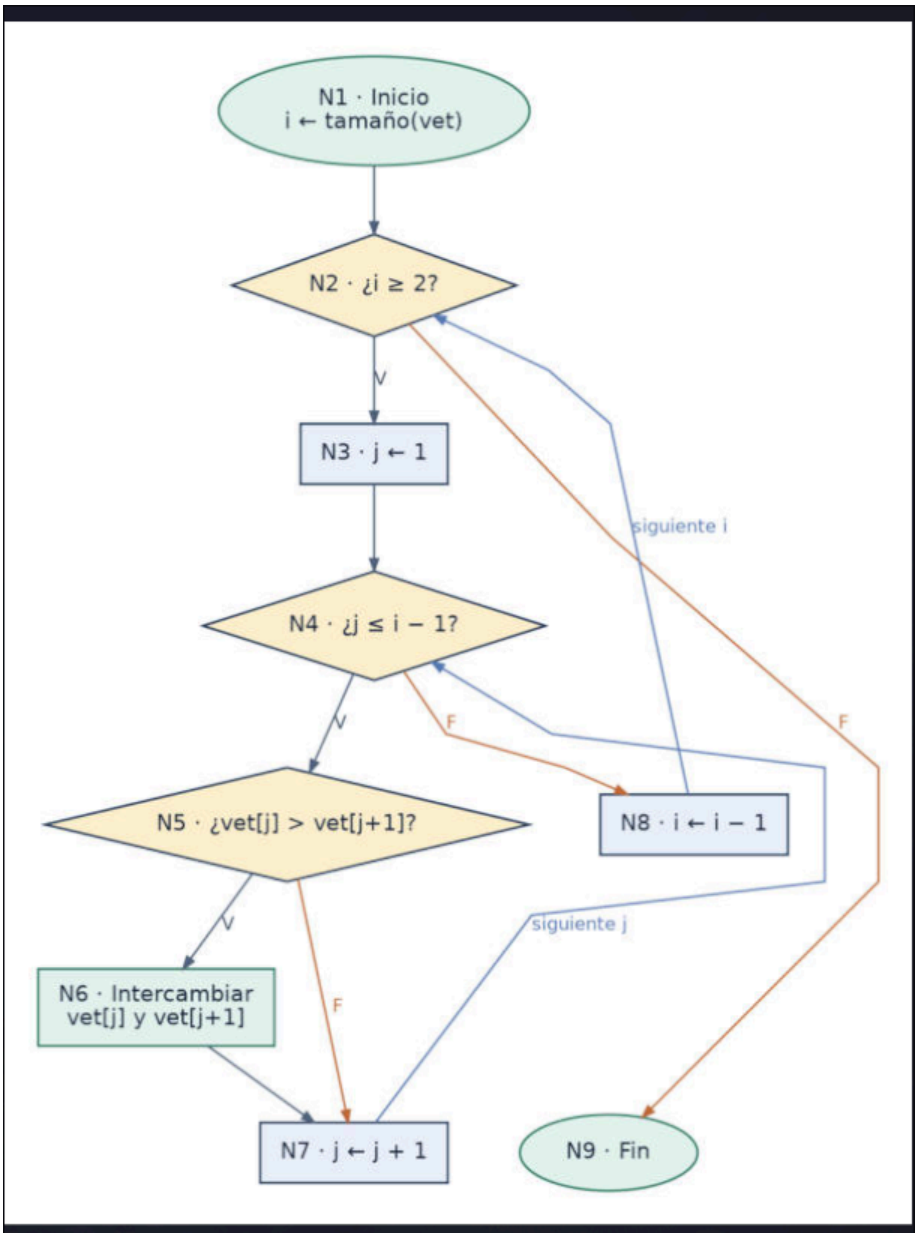
2026

1. Código base y criterio de interpretación

```
bubbleSort(int vet[]) {
   integer i, j, aux;
   for (i=tamanho(vet); i>=2; i--) {
      for (j=1; j<=i-1; j++) {
         if (vet[j] > vet[j+1]) {
            aux=vet[j];
            vet[j]=vet[j+1];
            vet[j+1]=aux;
         }
      }
   }
}
```

2. Actividad 1

El grafo distingue las tres condicionales (rombos) y los dos retornos de los bucles. V representa una evaluación verdadera y F una falsa. **Se identifican nueve nodos y once aristas visibles.**



NODO	OPERACIÓN	NODO	OPERACIÓN
N1	Inicio; i ← tamaño(vet)	N2	i ≥ 2?
N3	j ← 1	N4	j ≤ i − 1?
N5	vet[j] > vet[j+1]?	N6	Intercambiar
N7	j ← j + 1	N8	i ← i − 1
N9	Fin		

Aristas: N1→N2; N2→N3/N9; N3→N4; N4→N5/N8; N5→N6/N7; N6→N7; N7→N4; N8→N2.

3. Actividad 2

Para un único componente conexo del grafo de flujo de control se emplea V(G) = E − N + 2P.

VARIABLE	VALOR	JUSTIFICACIÓN
E (aristas)	11	Conexiones dirigidas entre nodos
N (nodos)	9	N1, N2, …, N9
P (componentes)	1	Un único subgrafo conectado
V(G)	11 − 9 + 2(1) = 4	Resultado de complejidad ciclomática

existen tres nodos de decisión, N2, N4 y N5; por tanto V(G) = 3 + 1 = 4.

4. Actividad 3

Se elige cobertura de decisiones + condición simple. Como cada decisión contiene un único predicado booleano, basta con obtener V y F en los nodos N2, N4 y N5. Los cuatro caminos se ejecutan con entradas factibles.

CAMINO	RASGO QUE LO DISTINGUE	ENTRADA
CP1	Salida directa N2→N9; no entra al for externo	[7]
CP2	Incluye N5→N7 (condición de intercambio F)	[1, 2]
CP3	Incluye N5→N6 (intercambio V)	[2, 1]
CP4	Repite el ciclo interno N7→N4 y externo N8→N2	[2, 3, 1]

Secuencias observadas de nodos

CP1 N1 → N2(F) → N9

CP2 N1 → N2(V) → N3 → N4(V) → N5(F) → N7 → N4(F) → N8 → N2(F) → N9

CP3 N1 → N2(V) → N3 → N4(V) → N5(V) → N6 → N7 → N4(F) → N8 → N2(F) → N9

CP4 N1 → N2(V) → N3 → N4(V) → N5(F) → N7 → N4(V) → N5(V) → N6 → N7 → N4(F) → N8 → N2(V) → N3 → N4(V) → N5(V) → N6 → N7 → N4(F) → N8 → N2(F) → N9

5. Actividad 4d

Las entradas y salidas de la tabla fueron verificadas con una implementación ejecutable del algoritmo. Los caminos corresponden al grafo de la figura 1.

ID CP	CAMINO EN EL GRAFO	VALORES DE PRUEBA	RESULTADO ESPERADO
CP1	N1→N2(F)→N9	[7]	[7]
CP2	N1→N2(V)→N3→N4(V)→N5(F)→N7→N4(F)→N8→N2(F)→N9	[1, 2]	[1, 2]
CP3	N1→N2(V)→N3→N4(V)→N5(V)→N6→N7→N4(F)→N8→N2(F)→N9	[2, 1]	[1, 2]
CP4	N1→N2(V)→N3→N4(V)→N5(F)→N7→N4(V)→N5(V)→N6→N7→N4(F)→N8→N2(V)→N3→N4(V)→N5(V)→N6→N7→N4(F)→N8→N2(F)→N9	[2, 3, 1]	[1, 2, 3]

6. Comprobación de cobertura y ejecución

DECISIÓN	SALIDAS OBSERVADAS	COBERTURA
N2: $i \geq 2$	V y F	2/2
N4: $j \leq i - 1$	V y F	2/2
N5: vet[j] > vet[j+1]	V y F	2/2
Total	6 de 6 salidas	100 %

Ejecución de ocho pruebas unitarias con unittest: CP1–CP4 y cuatro pruebas complementarias (lista vacía, duplicados, números negativos/cero y orden inverso). Resultado: 8 pruebas ejecutadas; 8 exitosas; 0 fallidas.

```python
class TestRegresion(unittest.TestCase):
    def test_lista_vacia(self):
        datos = []
        bubble_sort(datos)
        self.assertEqual(datos, [])

    def test_repetidos(self):
        datos = [2, 2, 1, 1]
        bubble_sort(datos)
        self.assertEqual(datos, [1, 1, 2, 2])

    def test_valores_negativos_y_cero(self):
        datos = [0, -3, 5, -1]
        bubble_sort(datos)
        self.assertEqual(datos, [-3, -1, 0, 5])

    def test_orden_inverso(self):
        datos = [5, 4, 3, 2, 1]
        bubble_sort(datos)
        self.assertEqual(datos, [1, 2, 3, 4, 5])


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

```
test_CP1_sin_entrar_al_bucle_externo (test_bubble_sort.TestCamin
test_CP2_comparacion_sin_intercambio (test_bubble_sort.TestCamin
test_CP3_comparacion_con_intercambio (test_bubble_sort.TestCamin
test_CP4_repeticion_de_bucles (test_bubble_sort.TestCaminosIndep
test_lista_vacia (test_bubble_sort.TestRegresion.test_lista_vaci
test_orden_inverso (test_bubble_sort.TestRegresion.test_orden_in
test_repetidos (test_bubble_sort.TestRegresion.test_repetidos) .
test_valores_negativos_y_cero (test_bubble_sort.TestRegresion.te

----------------------------------------------------------------------
Ran 8 tests in 0.001s

OK
```